Operation Runbook

Music Podcast Player

1 Introduction

This operations runbook explains the basic steps for running, checking, troubleshooting, and maintaining the Music Podcast Player application. It is meant to help developers quickly understand what to do when starting the system, checking if services are working, or fixing common problems.

2 System Overview

Music Podcast Player is made up of a frontend, backend API, database service, PostgreSQL database, and supporting local services. The frontend is the web interface that users interact with. The backend handles requests from the frontend, connects to the database service, and communicates with external services such as the Podcast Index API.

Component	Purpose
Frontend	Web interface for the application
Backend API	Handles frontend requests
Database Service	Connects the backend to PostgreSQL
PostgreSQL	Stores users, playlists, and track data
Migrations	Sets up and updates the database schema
MinIO	Local S3-compatible storage
DynamoDB	Local rate-limit token storage

Table 1: System Components

3 Local Startup and Shutdown

The application can be started locally using Docker Compose from the project root folder.

3.1 Startup

First, open a terminal in the project root. The root folder should contain `docker-compose.yml`. Then run:

```
docker compose up
```

If the containers need to be rebuilt, run:

```
docker compose up --build
```

After startup, the frontend should be available at:

`http://localhost:5173`

The backend API should be available at:

`http://localhost:5000`

3.2 Shutdown

To stop the local application, run:

```
docker compose down
```

To reset local data and remove Docker volumes, run:

```
docker compose down -v
```

The reset command should be used carefully because it removes local database data.

4 Service Health Checks

After the system starts, the main services should be checked to make sure they are running correctly.

- Use `docker compose ps` to confirm containers are running.
- Open `http://localhost:5173` to confirm the frontend loads.
- Check backend logs with `docker compose logs backend-api-service`.
- Check database logs with `docker compose logs database`.
- Check migration logs with `docker compose logs migrations`.

If the frontend loads and the backend does not show repeated errors, the local system is likely running correctly.

5 Common Issues and Fixes

5.1 Migration Failure

If the migration service fails, check the migration and database logs:

```
docker compose logs migrations
docker compose logs database
```

For local development, the common fix is to reset the environment:

```
docker compose down -v
docker compose up --build
```

5.2 Database Fails to Start

If the database container fails, check the database logs:

```
docker compose logs database
```

If the issue is caused by old local data, reset the environment with:

```
docker compose down -v
docker compose up --build
```

5.3 Frontend Cannot Connect to Backend

If the frontend loads but API requests fail, confirm that the backend container is running:

```
docker compose ps
docker compose logs backend-api-service
```

The frontend should connect to the backend at:

```
http://localhost:5000
```

If this value is wrong, check the frontend runtime configuration file, such as `config.json`.

5.4 Unauthorized Requests

A `401 Unauthorized` response usually means the user is not logged in or the request needs authentication. The user should refresh the page, log in again, and confirm that the account exists. Backend logs can also be checked for authentication errors.

5.5 Playlist Track Information Missing

If a playlist item shows missing information, such as "track title not found," the saved track object may be missing required fields. The track should include information such as `title`, `track_url`, `guid`, and `duration`. The issue may be in the frontend feed parser, playlist API helper, or backend playlist route.

6 Testing and Quality Checks

Before merging or deploying changes, the team should run basic frontend and backend checks.

6.1 Frontend

From the frontend folder, run:

```
npm install
npm run test -- --run
npm run lint
npm run format-check
npm run build
```

If formatting needs to be fixed, run:

```
npm run format-fix
```

6.2 Backend

From the backend package folders, run:

```
python -m pytest
python -m ruff check
python -m ruff format --check
```

If formatting needs to be fixed, run:

```
python -m ruff format
```

7 Deployment Checks

The project uses GitHub Actions for testing and deployment. Before deploying, the team should confirm that the correct branch is being used and that the workflow checks pass.

For a development deployment, changes are usually pushed to the `dev` branch. For production, the team should make sure the development version works first, pull requests are reviewed, and all required AWS environment variables are configured.

Before deployment, check the following:

- Frontend checks pass.
- Backend checks pass.
- Docker Compose works locally.
- GitHub Actions workflows pass.
- No secrets or private credentials were committed.
- Database migrations are reviewed before production deployment.

8 Monitoring and Incident Response

For deployed environments, the team should monitor GitHub Actions and AWS CloudWatch. Important items to watch include backend errors, failed deployments, database connection errors, authentication failures, and external API failures.

If the website is down, first check whether the frontend deployment completed successfully. Then check backend logs, API errors, and recent commits. If the backend is failing, review logs, environment variables, database access, and external service connections.

If the database is having problems, check database logs, migration logs, and connection settings. For serious production issues, the team should consider restoring from a known good backup only after identifying the cause.

9 Security and Secrets

Sensitive values should not be committed to the repository. Local secrets should stay in local secret files, and production secrets should be stored through AWS services.

Examples of sensitive values include:

- Podcast Index API keys
- JWT secret keys
- Database passwords
- AWS access keys

If a secret is accidentally committed, it should be rotated immediately.

10 Routine Maintenance Checklist

Before demos, major merges, or deployments, the team should complete the following checklist:

- Start the app locally with Docker Compose.
- Confirm the frontend loads.
- Confirm the backend API is running.
- Confirm the database is healthy.
- Test login or signup.
- Test RSS feed loading or search.
- Test music playback.
- Test playlist features.
- Run frontend and backend checks.
- Check GitHub Actions results.

11 Quick Reference Commands

Task	Command
Start app	`docker compose up`
Rebuild app	`docker compose up --build`
Stop app	`docker compose down`
Reset local data	`docker compose down -v`
Check containers	`docker compose ps`
View all logs	`docker compose logs`
Backend logs	`docker compose logs backend-api-service`
Database logs	`docker compose logs database`
Migration logs	`docker compose logs migrations`

Table 2: Common Operations Commands

12 Conclusion

This runbook gives the team a basic process for operating Music Podcast Player. It explains how to start the app, check service health, troubleshoot common issues, run quality checks, and respond to deployment or production problems. The main goal is to make routine operations clear and easy to follow.